International Institute of Information Technology, Bangalore

Software Production Engineering Mini Project Report
Scientific Calculator

Submitted By

Meenal Hirwani (MT2025071)

In the guidance of Prof. B. Thangaraju

Abstract:

This project demonstrates the implementation of a DevOps-based CI/CD pipeline for automating the build, testing, containerization, and deployment of a web application. The application is developed using Python and HTML, with GitHub used for version control. Jenkins is used to automate the CI/CD pipeline, while Docker is used to containerize the application for consistent execution across environments. Ansible is used for automated deployment and configuration management.

The project shows how DevOps tools can be integrated to automate the software development lifecycle, reduce manual effort, and enable faster and more reliable application deployment.

DevOps:

What is DevOps? DevOps is a software development approach that integrates Development (Dev) and Operations (Ops) teams to improve collaboration, automation, and efficiency in the software development lifecycle.

It focuses on continuous integration, continuous testing, continuous deployment, and monitoring, allowing teams to build, test, and release software faster and more reliably. DevOps uses tools and automation to reduce manual work and ensure smooth communication between development and operations teams.

Advantages of DevOps:

- Faster Software Delivery: DevOps enables continuous integration and deployment, allowing applications to be delivered quickly.
- Improved Collaboration: Development and operations teams work together, improving communication and reducing conflicts.
- Reduced Deployment Failures: Frequent small updates reduce the risk of major failures.
- Automation of Processes: Tasks such as building, testing, and deployment are automated, reducing manual effort and errors.
- Automation of Processes: Tasks such as building, testing, and deployment are automated, reducing manual effort and errors.

Tools and Technologies Used:

Programming Language:

Python Version Control:

Git & GitHub Build Automation:

FastApi CI/CD Pipeline:

GitHub Actions / Jenkins Containerization:

Docker Testing Framework: Pytest

Source Code Management:

SCM tracks changes made to a source code repository. SCM monitors a code base's modifications and assists in resolving disputes when combining updates from several authors. Version Control is another name for SCM. The local machine (repository) is where the complete project is initially built. After that, this project gets uploaded to GitHub's remote repository.

To push the local repository to the remote repository, a set of instructions is run.
git init

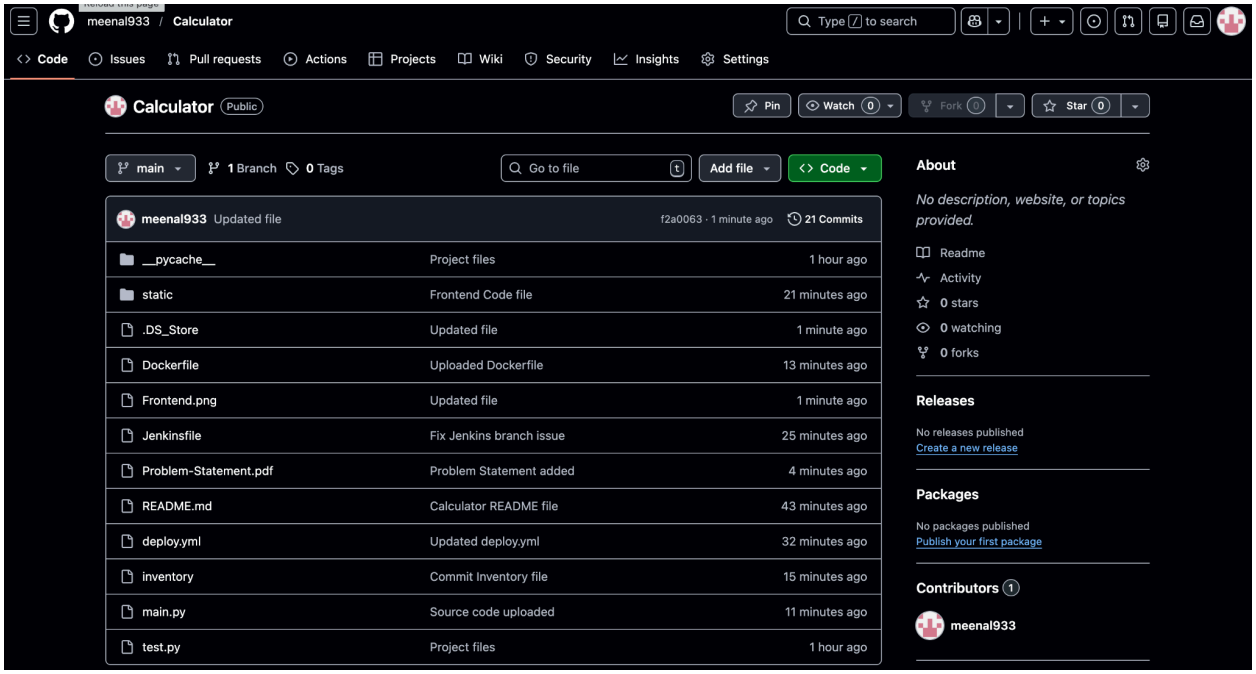
git status

git add

git commit -m “Commit Message”

git remote add origin https://github.com/meenal933/Calculator.git

git push -u origin main



Source code Management

Docker:

Docker is an open-source platform used to develop, package, and run applications in containers. Containers allow applications to run in a consistent and isolated environment, along with all the necessary dependencies and libraries.

Docker helps developers avoid issues related to differences between development and production environments. By using Docker, applications can be easily deployed and run on any system that supports Docker.

The following command can be used to build the Docker image:

docker build -t <USERNAME>/<IMAGE NAME> : <TAG>

To push the docker image use command:

docker push <USERNAME>/<REPOSITORY_NAME>:tagname

The docker image is pushed from jenkins after the build stage and pulled on from ansible job. The docker image can be run using command

docker run –i –t . After pushing the created docker image to Docker Hub, we can check on our Docker hub profile.

```
# Use python base image
FROM python:3.9-alpine

WORKDIR /app

COPY . .

RUN pip install --no-cache-dir fastapi uvicorn pytest

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile

Jenkins:

Jenkins is an open-source automation server used to implement Continuous Integration (CI) and Continuous Deployment (CD) in software development. It helps automate tasks such as building, testing, and deploying applications, which reduces manual effort and improves development efficiency.

In a DevOps workflow, Jenkins integrates with tools like GitHub, Docker, and Ansible to automatically trigger pipelines whenever code changes are pushed to the repository. This ensures that the application is continuously tested and deployed without manual intervention.

Key Features

- Automation: Automates build, test, and deployment processes.
- CI/CD Pipeline: Supports continuous integration and continuous delivery.
- Plugin Support: Provides many plugins to integrate with different DevOps tools.
- Integration: Works with tools like Git, Docker, Ansible, and cloud platforms.

Jenkins helps teams deliver software faster and more reliably by automating the software development lifecycle.

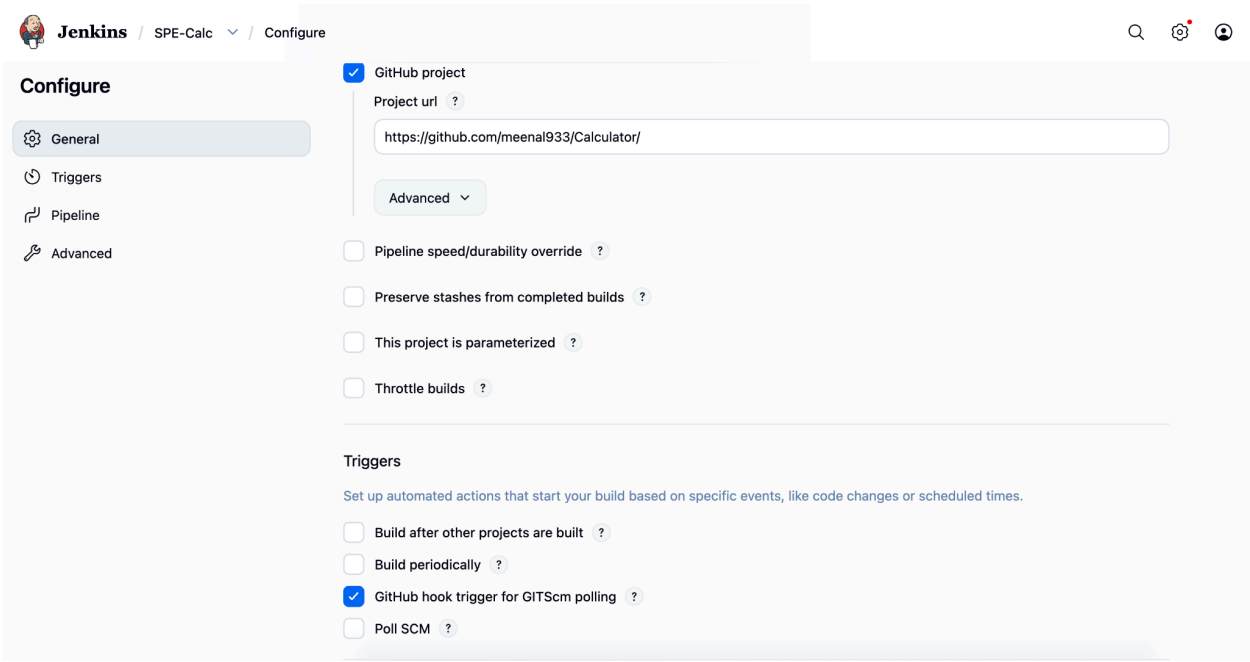
Configuration of Project in jenkins:

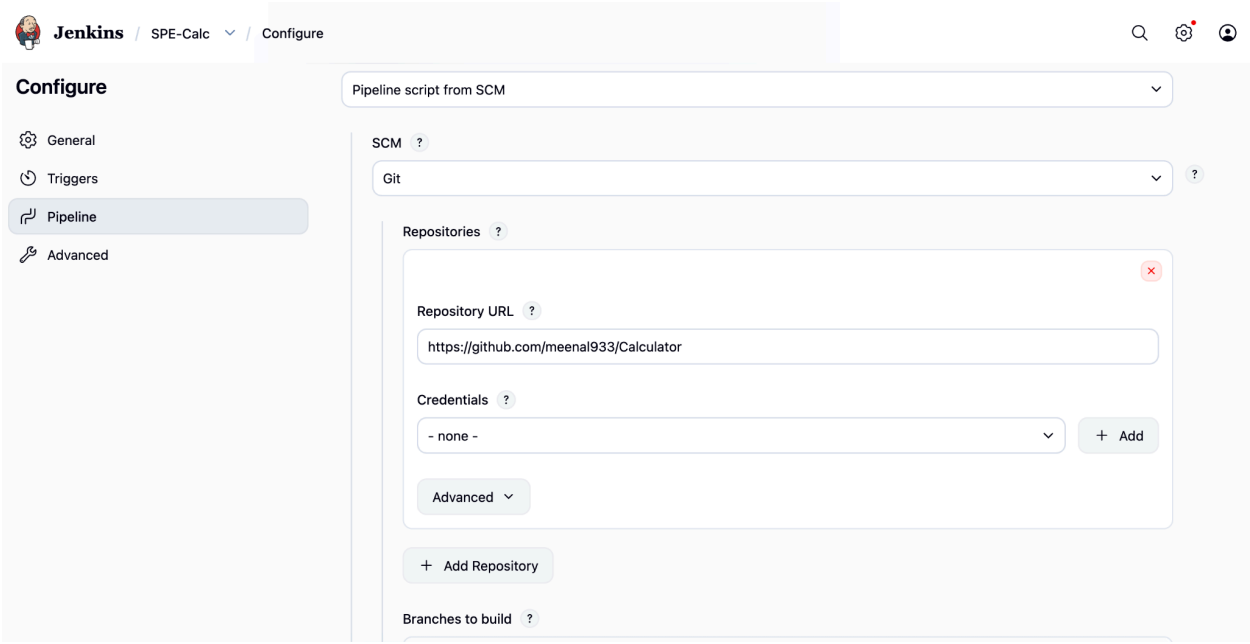
To run Jenkins in local machine there is a command to run it

sudo systemctl start jenkins

To check status of Jenkins:

sudo systemctl status jenkins





Configuration of Project

Here, we created a github project in jenkins and gave the URL of our github repository and specified the jenkins file path in configuration. Set System Credentials for Docker Hub and Local Machine for Ansible playbook, here we run project in local machine and pull the image from docker image so we have to specify that system configuration in jenkins system credentials and add two credentials one for Docker Hub Repository and other one for Local Machine. To set DockerHub credentials we have to specify the username of dockerhub from where we have to pull the image.

Ansible:

One tool for continuous deployment is Ansible. where the deliverables are distributed on what are known as managed nodes, which have a single control node. Only the control node has to have Ansible installed. Ansible uses SSH to establish a connection with the controlled nodes. Ansible provides a collection of commands in the form of a yml file that must be run on the managed node. We refer to it as a playbook. We offer details on the managed nodes in addition to the playbook. It is known as inventory.

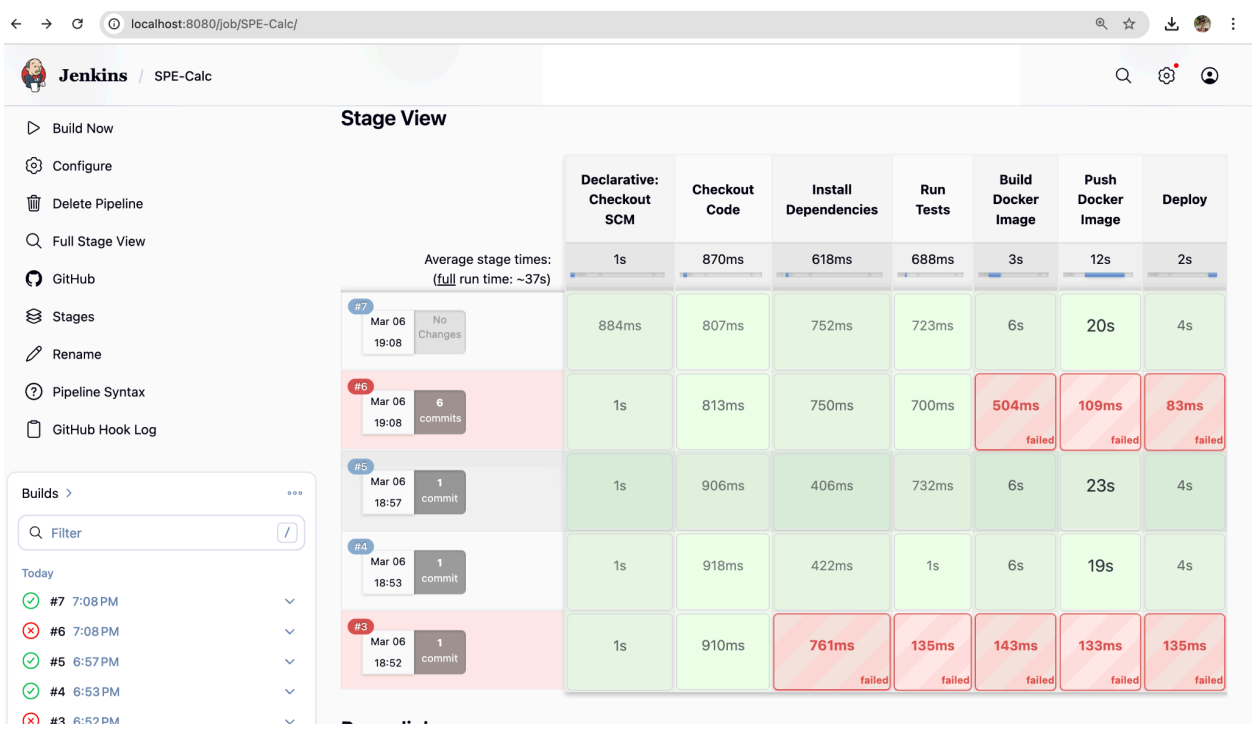
To execute this playbook we run the following command:

ansible-playbook -i <playbook> -i <inventory>

The hosts and host groups that a playbook's commands, modules, and tasks run on are specified in the Ansible inventory file. Depending on your Ansible environment and plugins, the file may be in one of several formats. The configuration, deployment, and orchestration functions of Ansible are documented and carried out by the Ansible playbooks. To launch the application's docker image, we must install Python pip on the host in addition to docker. The container is launched on the designated hosts when the docker image is downloaded from the docker hub. The Ansible playbook handles this process.

Run Project as Pipeline in Jenkins:

To run the project as a pipeline in Jenkins we have to click build now to see each stage of the pipeline and show the output of each stage.



Scientific Calculator Source code:

main.py 5

Users > meenalhirwani > Documents > Mini-Project > main.py > ...

```python
from fastapi import FastAPI
import math
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse

app = FastAPI()

# Allow CORS for frontend interaction!
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# app.mount("/static", StaticFiles(directory="static"), name="static")

@app.get("/")
def home():
    return FileResponse("static/index.html")

@app.get("/sqrt/{x}")
def square_root(x: float):
    if(x<0): return {"result":"Enter Valid number"}
    return {"result": math.sqrt(x)}

@app.get("/factorial/{x}")
def factorial(x: int):
    if(x<0): return {"result": "Enter Valid number"}
    return {"result": math.factorial(x)}

@app.get("/ln/{x}")
def natural_log(x: float):
    if(x<0): return {"result":"Enter Valid number"}
    return {"result": math.log(x)}

@app.get("/power/{x}/{b}")
def power(x: float, b: float):
    return {"result": math.pow(x, b)}
```

Adding Test Cases:

Unit Testing is a software testing technique in which the smallest testable parts of a software application, called units, are tested individually to verify that they function correctly. A unit may refer to a single function, method, class, or module of a program. The purpose of unit testing is to ensure that each component of the software performs as expected before integrating it with other components.

In this project, unit testing is performed using **pytest** to test the individual functions of the scientific calculator. The tests verify that mathematical operations return correct

results before the application is built into a Docker image and deployed through Jenkins. This helps detect errors early and ensures the reliability of the application.

test_main.py 2

Users > meenalhirwani > Documents > Mini-Project > test_main.py > ...

```python
import pytest
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_square_root():
    response = client.get("/sqrt/-9")
    assert response.status_code == 200
    assert response.json()["result"] == "Enter valid number"
    response = client.get("/sqrt/9")
    assert response.status_code == 200
    assert response.json()["result"] == 3

def test_factorial():
    response = client.get("/factorial/5")
    assert response.status_code == 200
    assert response.json()["result"] == 120
    response = client.get("/factorial/-5")
    assert response.status_code == 200
    assert response.json()["result"] == "Enter valid number"

def test_natural_log():
    response = client.get("/ln/1")
    assert response.status_code == 200
    assert response.json()["result"] == 0.0

def test_power():
    response = client.get("/power/2/3")
    assert response.status_code == 200
    assert response.json()["result"] == 8.0
```

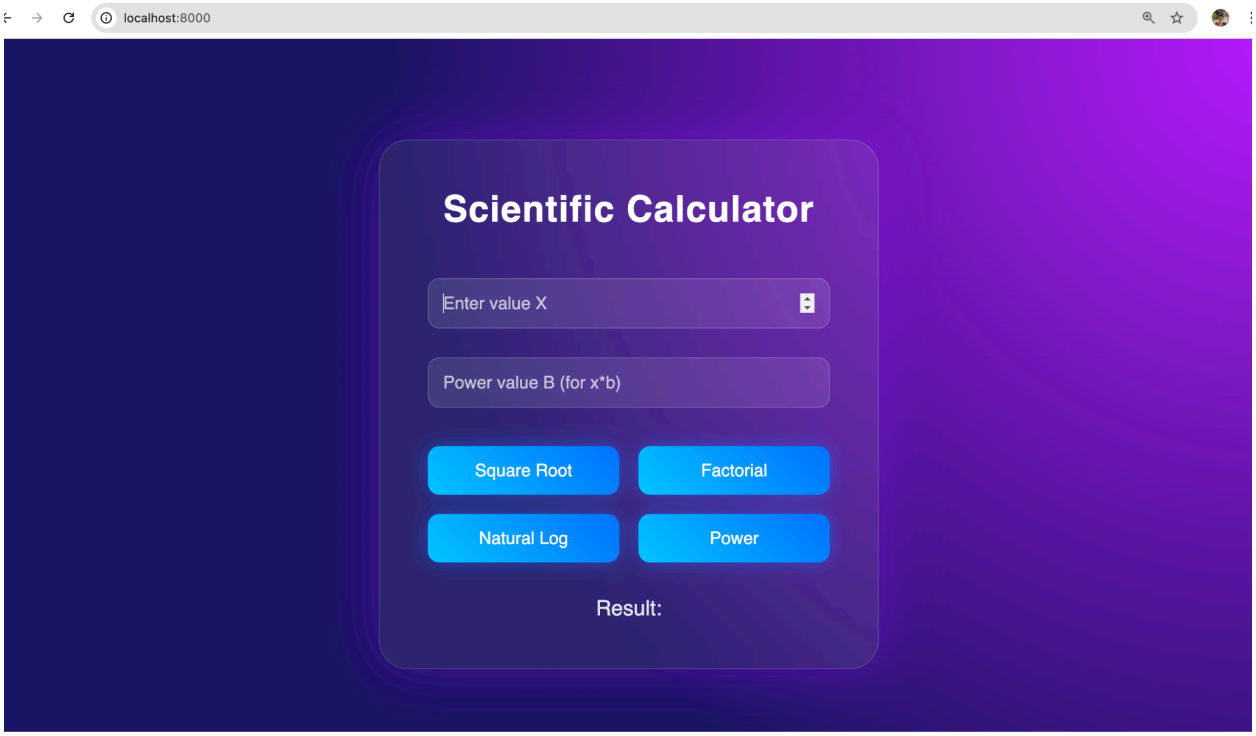
Pytest:

pytest is a popular testing framework for Python. It is used to write simple as well as complex tests with ease. It supports features like fixtures, parameterized testing,

assertions with better error reporting, and plugins for extended functionality. It's widely used for unit, functional, and integration testing. To run tests, you just need to write functions starting with test_ and execute them using the pytest command.

Run Project:

To run the project first run the pipeline in the jenkins. The project runs successfully now our docker image is pulled successfully from the docker hub and runs in the local machine to check if it runs or not. We can access it on local machine using localhost:8000



Github repo: https://github.com/meenal933/Calculator

Dockerhub Repo: https://hub.docker.com/r/meenal933/scientific_calculator